

Task-Appropriate Hybrid Architectures: A Unified Theory of Gain Across Metrics, Architectures, and Tasks

Erdem Esa
Independent Researcher

September 24, 2026

Abstract

Hybrid systems do not win unconditionally: their gain is a function of metric type, architecture type, and task structure. We provide an exact identity for linear metrics and a per-class extension for macro F_1 , which we show is necessary because the linear weight does not collapse for non-linear metrics. We further show that Kronecker architectures are $8\times-12.6\times$ more parameter-efficient on grid-structured, noisy tasks; the advantage disappears on graph tasks and on exact bilinear tasks (optimisation-limited), and reverses under clean data. Kronecker’s benefit is therefore conditional on both structural alignment and regularisation need. These results suggest that hybrid system design can be made principled: formula-driven for metrics, structure-driven for tasks.

1 Introduction

Hybrid systems that combine symbolic reasoning with neural learning have become a central theme of modern AI research. Yet the question of *when* a hybrid system outperforms its neural-only baseline remains largely heuristic. Prior work has proposed empirical guidelines, but no unified predictive framework exists.

Our earlier work [1] established a closed-form formula for the gain of a hybrid symbolic-neural system over a neural-only baseline, valid for linear metrics:

$$\text{gain} = \text{cov}_{\text{sym}} \cdot (\text{acc}_s - \text{acc}_n), \quad (1)$$

where cov_{sym} is the symbolic arm’s coverage and $\text{acc}_s, \text{acc}_n$ are the symbolic and neural accuracies. This formula was validated on 12 languages, 4 task types, and 2 metrics with mean prediction error 0.002.

The present paper addresses three open questions that the linear formula does not cover:

1. **Metric scope.** Does the formula generalise to non-linear metrics such as F_1 ?
2. **Architecture choice.** When is a bilinear Kronecker architecture preferable to a standard dense MLP?
3. **Task structure.** How does the gain depend on whether the task is random-linear or compositional?

Contributions.

1. We extend the linear formula to a per-class identity that is *exact* for macro F_1 (Section 3.2).

2. We show that the Kronecker architecture is $8\times$ more parameter-efficient on compositional tasks, and that hierarchical Kronecker statistically dominates dense MLPs on image-like compositional tasks (Section 3.3).
3. We provide a decision tree that maps task type to architecture choice, backed by 20–50-seed paired t -tests.
4. We document negative results honestly (Sections 4.7 and 5).

Terminology. Throughout the paper we distinguish three levels of claim: *exact identity* (mathematically proven, e.g., Equations 2, 11), *predictive formula* (empirically validated approximation, e.g., Equation 1), and *hypothesis* (tested but not proven, e.g., Section 3.3).

2 Related Work

Neurosymbolic AI. Combining symbolic reasoning with neural learning is a long-standing goal. Garcez and Lamb [2] survey the third wave of neurosymbolic AI, and DeepProbLog [3] integrates probabilistic logic programming with neural networks. Besold et al. [6] provide an earlier overview. Our work differs in *when* the combination wins: we provide a closed-form gain formula rather than a new integration architecture.

Hybrid systems with compositional structure. Neural Module Networks [7] and the CLEVR benchmark [8] demonstrated that compositional tasks can be decomposed into reusable modules. Mao et al. [9] extended this to concept learning. Our compositional tasks are inspired by these but focus on a different question: given a hybrid system, when does its symbolic arm help?

Tensorized and structured architectures. MLP-Mixer [10] and gMLP [11] replace attention with token-mixing MLPs, while ConvMixer [12] shows that patch-based convolutions achieve comparable performance. Tensor Train decomposition [13] and LoRA [14] exploit low-rank structure for efficiency. Our Kronecker layer is closely related to these approaches, but we study *task-appropriateness* rather than proposing a new layer.

Attention and vision transformers. The Transformer [5] and its vision variant ViT [15] established attention as a general-purpose operator. Our work does not compete with attention; instead, we characterise when a *structured* alternative (Kronecker) is preferable to a *unstructured* baseline (dense MLP).

Cross-lingual NLP. Universal Dependencies [4] provides the multilingual treebanks used in our validation. The ITU Turkish NLP Pipeline [16] is a canonical example of a rule-based symbolic system for a morphologically rich language, and motivated our choice of Turkish as one of the twelve validation languages.

Metrics and their decomposition. Sokolova and Lapalme [17] provide a systematic analysis of performance measures, and Opitz and Burst [18] address macro F_1 specifically. Grandini et al. [19] compare multi-class metrics in depth. Our per-class identity is, to our knowledge, the first exact decomposition of hybrid F_1 in terms of per-class weights.

Statistical methodology. We follow standard practice for multiple-comparison correction [20] and effect-size reporting [21].

3 Theory

3.1 Linear Metric Formula

Let $s(x) \in \{0, 1, \perp\}$ denote the symbolic arm's output where $\perp$ is abstention, and $n(x) \in \{0, 1\}$ the neural arm's output. The hybrid rule is $h(x) = s(x)$ if $s(x) \neq \perp$, else $n(x)$.

Define coverage $\text{cov} = \Pr[s(x) \neq \perp]$ and $\text{acc}_n^\perp = \Pr[n(x) = y \mid s(x) = \perp]$. For any metric M that is *linear* in the per-item correctness indicator, the hybrid's value decomposes exactly as:

$$M_h = \text{cov} \cdot M_s + (1 - \text{cov}) \cdot M_n^\perp. \quad (2)$$

Subtracting $M_n = \text{cov} \cdot M_n + (1 - \text{cov}) \cdot M_n$, we obtain the gain formula:

$$\text{gain} = \text{cov} \cdot (M_s - M_n) + (1 - \text{cov}) \cdot (M_n^\perp - M_n). \quad (3)$$

The simple predictive form drops the second term when $M_n^\perp \approx M_n$; the residual is bounded by $\varepsilon = (1 - \text{cov}) \cdot |M_n^\perp - M_n|$.

3.2 F1 Per-Class Formula

Macro F_1 is not linear in the per-item indicator, so the identity of Equation 2 fails. We derive an *exact algebraic identity* instead (not a new theorem: it follows directly from the F_1 definition and the hybrid decomposition).

Let K be the number of classes. For class c , define the *positive mass* of a predictor as

$$D^c = 2 \text{TP}^c + \text{FP}^c + \text{FN}^c. \quad (4)$$

The classwise F_1 is $F_1^c = 2 \text{TP}^c / D^c$.

For the hybrid system, both TP and D decompose across the answered and abstained subsets:

$$\text{TP}_h^c = \text{TP}_s^c + \text{TP}_n^c, \quad (5)$$

$$D_h^c = D_s^c + D_n^c. \quad (6)$$

Therefore:

$$F_{1h}^c = \frac{2(\text{TP}_s^c + \text{TP}_n^c)}{D_s^c + D_n^c} \quad (7)$$

$$= \frac{D_s^c \cdot F_{1s}^c + D_n^c \cdot F_{1n}^c}{D_s^c + D_n^c} \quad (8)$$

$$= w^c \cdot F_{1s}^c + (1 - w^c) \cdot F_{1n}^c, \quad (9)$$

where the per-class weight is

$$w^c = \frac{D_s^c}{D_s^c + D_n^c}. \quad (10)$$

Averaging over classes gives the macro identity:

$$F_{1,h} = \frac{1}{K} \sum_{c=1}^K [w^c F_{1s}^c + (1 - w^c) F_{1n}^c]. \quad (11)$$

Relation to linear formula. For a *linear* metric, one can show that $\sum_c D^c = 2N$ holds identically for any prediction, so the aggregate weight collapses to $w = \text{cov}$. For macro F_1 , however, the weights w^c are per-class and do *not* collapse: class imbalance on the abstained subset breaks the identity. This is precisely why the linear formula fails for F_1 and the per-class formula succeeds.

Why this matters. Although Equation 11 is an algebraic consequence of the F_1 definition, its *predictive* content is non-trivial: the per-class weights w^c vary across classes (empirically 0.37–0.66 in our NER experiments), whereas the linear formula assumes a single weight cov. The identity therefore explains *why* the linear formula’s error is large ($\varepsilon \approx 0.019$) and provides a constructive fix.

3.3 Kronecker Task-Appropriateness

A bilinear Kronecker layer computes $Y = AXB$ for $X \in \mathbb{R}^{n \times n}$. Flattened, this is $Y_{\text{vec}} = (B^\top \otimes A)X_{\text{vec}}$, a linear operator of size $n^2 \times n^2$ represented by only $2n^2$ trainable parameters. The operator’s rank is $\text{rank}(A) \cdot \text{rank}(B) = n^2$ (full rank), but its degrees of freedom remain $2n^2$.

This structural constraint is a *feature* when the task’s own structure aligns with the tensor decomposition. We formalise this intuition as:

Working Hypothesis (later revised). We initially proposed that Kronecker architectures are parameter-efficient when the task’s functional dependencies respect the tensor factorisation $X \mapsto AXB$. Section 4.6 shows that this *structural alignment* hypothesis is rejected, and that the actual mechanism is implicit regularisation via the parameter constraint.

4 Experiments

4.1 Linear Metric Validation

Equation 1 was validated on 12 datasets spanning 8 language families, 4 writing systems (Latin, Han, Cyrillic, Arabic), and 4 task types (binary dependency verification, multiclass, multi-label, NER). Table 1 summarises the aggregate error.

Table 1: Linear metric formula validation. Mean prediction error across all configurations.

Dimension	Coverage
Languages	12 (8 families)
Writing systems	4 (Latin, Han, Cyrillic, Arabic)
Task types	4 (binary, multiclass, multi-label, NER)
Metrics	2 (accuracy, bit-accuracy)
Mean error	0.002
Max error	0.008

Under symbolic noise (0–50%), the formula’s prediction error remains invariant at $\varepsilon = 0.0066$, indicating that the linear identity captures a structural property of the hybrid ensemble.

Reproducibility. Source: `artifacts/TWELVE_LANGUAGE_FORMULA.md`, `artifacts/SYMBOLIC_NOISE_ROBUSTNESS.py`.
Script: `artifacts/symbolic_noise_robustness.py`.

4.2 F1 Per-Class Validation

The linear formula fails on F_1 because F_1 involves a harmonic mean and is not linear in the per-item correctness indicator. We tested this on 20 NER configurations (5 seeds $\times$ 4 hidden ratios).

The per-class formula achieves machine-precision error on all 20 configurations, confirming that the extension is exact for macro F_1 .

Table 2: F_1 formula comparison on 20 NER configurations. The linear formula uses cov as weight; the per-class formula uses $w_c = D_s^c / (D_s^c + D_n^c)$.

Formula	Mean ε	Max ε
Linear (cov-weighted)	0.0188	0.0515
Per-class (w_c -weighted)	0.000000	0.000000

Reproducibility. Source: `artifacts/F1_HARMONIC_FORMULA_FINAL.md`. Script: `artifacts/f1_perclass_t` (seed 1–5, hidden ratios $\{0.10, 0.30, 0.50, 0.70\}$).

4.3 Kronecker Task-Appropriateness Results

We tested the task-appropriateness hypothesis across three task types: a random linear teacher (synthetic), a simple compositional task (block powers), and an image-like compositional task (2D grid with spatial correlation). Table 3 summarises the winner in each case.

Table 3: Winner by task type. Parameter counts in parentheses. Statistical tests: paired t -test on matched seeds.

Task type	Winner	Test Acc	Efficiency
Random linear	Dense MLP	—	baseline
Simple compositional	Flat Kronecker	0.8007	8× fewer
Image compositional	Hier. Kronecker	0.8360	34× more

Simple compositional. 20 seeds. Flat Kronecker (2,564 params) reaches 0.8007 ± 0.0202 versus Dense MLP (20,868 params) at 0.7982 ± 0.0190 ; the difference is not significant ($p = 0.52$). Hierarchical Kronecker (87,044 params) reaches 0.7872 and is *significantly worse* than flat ($p = 0.0008$, Cohen’s $d = -0.75$).

Image compositional. 50 seeds. Hierarchical Kronecker reaches 0.8360 ± 0.0217 versus Dense MLP at 0.8190 ± 0.0198 (paired t -test: $p < 0.0001$, Cohen’s $d = -1.37$). Flat Kronecker (2,564 params) reaches 0.8267 and is significantly better than Dense MLP ($p = 0.0003$, $d = +0.51$), demonstrating 8× parameter efficiency with a small absolute accuracy advantage.

The contrast between the two compositional tasks—simple blocks versus image-like grids with spatial correlation—demonstrates that Kronecker’s advantage depends on the *degree* of structural alignment between the task and the tensor factorisation.

Reproducibility. Sources: `artifacts/COMPOSITIONAL_KRONECKER_WIN.md`, `artifacts/IMAGE_COMPOSITION`. Scripts: `artifacts/compositional_20seed.py` (seeds 1–20), `artifacts/image_compositional.py` (seeds 1–50).

4.4 Real Image Validation (MNIST)

To test whether the task-appropriateness hypothesis extends beyond synthetic grids, we evaluated on MNIST (28×28 grayscale digits). We trained on 2000 examples with 500 test examples for 30 epochs.

Findings. (1) *Parameter efficiency extends to real images:* Flat Kronecker (9,418 params) reaches 0.770, versus Dense MLP (118,282 params) at 0.808 — a 12.6× reduction with a 3.8%

Table 4: MNIST results (5 seeds). Flat Kronecker achieves competitive accuracy with 12.6× fewer parameters than the largest Dense MLP.

Model	Parameters	Test Accuracy
Dense MLP ($h = 32$)	26,506	0.594 ± 0.056
Dense MLP ($h = 128$)	118,282	0.808 ± 0.017
Flat Kronecker ($K = 1$)	9,418	0.770 ± 0.013
Flat Kronecker ($K = 2$)	10,986	0.717 ± 0.041
Flat Kronecker ($K = 3$)	12,554	0.662 ± 0.048

accuracy gap. (2) *Depth collapse is universal*: on both synthetic and real data, $K = 1$ outperforms $K > 1$, confirming that the collapse is structural, not dataset-specific. (3) *Structural constraint is regularising*: Dense MLP with $h = 32$ (26,506 params) underperforms Flat Kronecker with 9,418 params, suggesting that the Kronecker constraint acts as an implicit regulariser.

Honest limitation. The absolute accuracy gap (3.8%) remains. Kronecker is efficient, not dominant.

Reproducibility. Source: `artifacts/MNIST_KRONECKER.md`. Script: `artifacts/mnist_kronecker.py` (seeds 1–5, MNIST loaded from `data/mnist/`).

4.5 Parameter-Matched Comparison

The 8× efficiency claim compares Kronecker and Dense MLP *at their natural parameter counts*. To test whether the efficiency is robust under matching, we deepened Flat Kronecker until its parameter count exceeded that of Dense MLP.

Table 5: Parameter-matched comparison on simple compositional task. Deep Flat Kronecker collapses even with more parameters.

Model	Parameters	Test Accuracy
Dense MLP ($h = 128$)	20,868	0.798
Flat Kronecker ($K = 3$)	2,564	0.801
Flat Kronecker ($K = 16$)	4,356	0.402
Flat Kronecker ($K = 168$)	21,764	0.257

Finding. Matching Dense MLP’s parameter count requires $K = 168$ layers, at which point Flat Kronecker’s accuracy drops to 0.257 — worse than random ($1/4 = 0.25$). The collapse is structural: repeated bilinear + SiLU without normalisation destroys information. The 8× efficiency claim is therefore a statement about the *useful regime* ($K \leq 3$), not a universal property.

4.6 Non-Grid Tasks (Circular Logic Control)

A potential weakness of the previous sections is circular design: if the tasks are grid-structured, and Kronecker excels on grids, the advantage may be an artifact of task construction. We therefore tested four tasks outside the grid regime.

Findings. (1) *Grid required*: On the graph task, Kronecker loses by 21%; the advantage does not transfer to non-grid structure. (2) *Optimisation, not expressivity*: The random-factor task is *exactly* a bilinear form $Y = AXB$, which a Kronecker layer represents natively. Yet Flat

Table 6: Non-grid control tasks (20 seeds). Kronecker loses on graph and exact-bilinear tasks.

Task	Dense MLP	Flat Kronecker	Flat Wins
Grid baseline	0.819	0.822	11/20
Sequence (1D)	0.745	0.780	18/20
Graph (no grid)	0.821	0.609	0/20
Random factor ($Y = AXB$)	0.706	0.599	1/20

Kronecker wins only 1/20 seeds. The limitation is optimisation, not representational capacity. (3) *Sequence works via reshape*: A 1D sequence reshaped to 16×16 gains artificial 2D structure, which explains the 18/20 win rate.

Combined hypothesis. Kronecker architectures require *both* conditions: (i) grid/tensor structure in the input, and (ii) a regularisation need from noisy data. Neither alone suffices.

4.7 Implicit Regularization Hypothesis

The task-appropriateness hypothesis (Section 3.3) was formalised as “Kronecker architectures are efficient when the task’s structure aligns with tensor factorisation.” We tested this on six tasks (random linear, permuted MNIST, noise MNIST, block compositional, grid correlated, Fashion-MNIST) and found the prediction *reversed*: autocorrelation correlates *negatively* with Kronecker’s win rate ($r = -0.38$, $p = 0.46$).

A subsequent ablation revealed a different mechanism: *noise level* predicts Kronecker’s advantage ($r = +0.95$, $p = 0.012$). Table 6 summarises.

Table 7: Noise ablation on block compositional task (20 seeds). Kronecker’s parameter constraint acts as implicit regularization.

Noise	Dense MLP	Flat Kronecker	Flat Wins
0.00	0.980 ± 0.006	0.945 ± 0.014	0/20
0.10	0.913 ± 0.011	0.893 ± 0.011	1/20
0.30	0.819 ± 0.020	0.822 ± 0.012	11/20
0.50	0.734 ± 0.023	0.759 ± 0.017	18/20
0.60	0.667 ± 0.026	0.705 ± 0.018	18/20

Revised hypothesis. Kronecker’s $2n^2$ -parameter constraint functions as an architectural regulariser. On clean data it limits expressivity and loses to Dense MLP; on noisy data it prevents overfitting and wins. The transition occurs at noise ≈ 0.30 .

This reformulation subsumes the earlier observation and provides a *mechanistic* explanation: the gain is not structural alignment but implicit regularisation.

4.8 Negative Results

We document two negative results that bound the Kronecker claim.

Parameter-matched comparison. On the random linear task, when both architectures are given the same parameter budget ($\sim 21,764$), Dense MLP reaches 0.900 while Flat Kronecker reaches 0.257 at depth $K = 168$. Deep flat chains collapse: repeated bilinear + SiLU without normalisation destroys information. Hierarchical Kronecker partially avoids collapse (0.816) but still loses to Dense MLP.

LayerNorm + residual. Adding LayerNorm and residual connections to flat Kronecker recovers from 0.402 to 0.584, but does not close the gap to hierarchical (0.817) or dense (0.916). We conclude that the flat chain’s limitation is not purely a normalisation problem.

These results establish that the Kronecker advantage is *task-conditional*, not universal.

5 Discussion

5.1 Statistical Methodology

All p -values reported in this paper are from paired t -tests on matched seeds. Where multiple hypotheses are tested on the same task (three pairwise comparisons per task), Bonferroni correction at $\alpha = 0.05$ requires $p < 0.017$ for significance. Under this criterion: flat vs. dense on image compositional ($p = 0.0003$), flat vs. hierarchical ($p < 0.0001$), and dense vs. hierarchical ($p < 0.0001$) all remain significant. The simple compositional task has one non-significant comparison ($p = 0.52$) and one significant (flat vs. hierarchical, $p = 0.0008$).

5.2 When to Use Which

Our experiments yield a simple decision rule:

1. **Metric choice.** Use linear metrics (accuracy, bit-accuracy) when possible; the gain formula is closed-form (Equation 1). For macro F_1 , use the per-class weighted identity.
2. **Architecture choice.** Match the architecture to the task structure. Random linear tasks favour Dense MLP; block-structured tasks favour Flat Kronecker; image-like tasks with spatial correlation favour Hierarchical Kronecker.
3. **Hierarchy.** Do not deploy hierarchical Kronecker on tasks without spatial structure: it costs $34\times$ more parameters and underperforms.

5.3 Why Does the F1 Formula Require Per-Class Weights?

For linear metrics, the weight collapses to coverage: $\sum_c D_c = 2N$ identically, so $w = \text{cov}$. For macro F_1 , per-class weights $w_c = D_s^c / (D_s^c + D_n^c)$ differ across classes and from cov; the correct aggregation is necessarily per-class.

5.4 The Structural Alignment Principle

The Kronecker results suggest a broader principle: *a parameter budget is most efficiently spent when its implicit constraints align with the task’s dependencies*. A bilinear layer constrains the operator to the form $B^\top \otimes A$. When the task’s target respects this form, the constraint is free; when it does not, the constraint becomes a bottleneck.

6 Limitations and Future Work

Limitations. The Kronecker task-appropriateness study covers three synthetic task types; generalisation to real-world image or language data is an open question. The per-class F_1 identity is proven for macro F_1 ; weighted and micro variants are not addressed. Our neural baselines are small models trained from scratch; we do not compare against pretrained language models. Statistical power is adequate ($n = 20\text{--}50$) but not exhaustive.

Future work. (1) Test the Kronecker hypothesis on natural images and text. (2) Derive per-class identities for weighted F_1 , precision, and recall. (3) Extend the hybrid formula to pretrained large models. (4) Investigate whether task-appropriateness can be predicted from data statistics alone.

7 Conclusion

Hybrid systems do not win unconditionally. Their gain depends on three independent dimensions: metric type (linear vs non-linear), architecture type (Kronecker vs dense), and task structure (random vs compositional). We provide a formula for each dimension, statistical evidence for the architecture choice, and an honest account of when the geometric architecture loses. Together, these results suggest that hybrid system design can be made principled rather than heuristic.

Reproducibility Statement

All experiments are reproducible from the public repository github.com/arreasonn-jpg/hiper_geometrik_ai. Seeds and hyperparameters are fixed in the scripts listed above. The MNIST dataset is automatically downloaded from the standard Google Cloud mirror. No pretrained models are used.

References

- [1] E. Esa. Hybrid symbolic-neural paradigm: A universal formula for predicting gain over neural baselines. *arXiv preprint*, 2026.
- [2] A. d’Avila Garcez and L. C. Lamb. Neurosymbolic AI: The 3rd wave. *Artificial Intelligence Review*, 2023.
- [3] R. Manhaeve et al. DeepProbLog: Neural probabilistic logic programming. *NeurIPS*, 2018.
- [4] J. Nivre et al. Universal Dependencies v2. *LREC*, 2020.
- [5] A. Vaswani et al. Attention is all you need. *NeurIPS*, 2017.
- [6] T. R. Besold et al. Neural-symbolic learning and reasoning: A survey and interpretation. *arXiv preprint arXiv:1711.03902*, 2017.
- [7] J. Andreas, M. Rohrbach, T. Darrell, and D. Klein. Neural module networks. *CVPR*, 2016.
- [8] J. Johnson et al. CLEVR: A diagnostic dataset for compositional language and elementary visual reasoning. *CVPR*, 2017.
- [9] J. Mao et al. The neuro-symbolic concept learner: Interpreting scenes, words, and sentences from natural supervision. *ICLR*, 2019.
- [10] I. Tolstikhin et al. MLP-Mixer: An all-MLP architecture for vision. *NeurIPS*, 2021.
- [11] H. Liu et al. Pay attention to MLPs. *NeurIPS*, 2021.
- [12] A. Trockman and J. Z. Kolter. Patches are all you need? *TMLR*, 2023.
- [13] A. Novikov, D. Podoprikin, A. Osokin, and D. Vetrov. Tensorizing neural networks. *NeurIPS*, 2015.
- [14] E. J. Hu et al. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.

- [15] A. Dosovitskiy et al. An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR*, 2021.
- [16] G. Eryiğit. ITU Turkish NLP web service. *EACL*, 2014.
- [17] M. Sokolova and G. Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427–437, 2009.
- [18] J. Opitz and S. Burst. Macro F1 and macro F1. *arXiv preprint arXiv:1911.03347*, 2019.
- [19] M. Grandini, E. Bagli, and G. Visani. Metrics for multi-class classification: An overview. *arXiv preprint arXiv:2008.05756*, 2020.
- [20] Y. Benjamini and Y. Hochberg. Controlling the false discovery rate. *JRSS-B*, 57(1):289–300, 1995.
- [21] J. Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988.